Deep Learning Assignment 2: Comparative Analysis of Pancake vs Tower Neural Networks for Quick Draw Classification

Aitsam Ul Haq
25280072

March 1, 2026

GitHub Repository: `https://github.com/Aitsam-Ul-Haq/AI-QuickDraw-Deep-vs-wide-Model/tree/main/DL_PA2`

1 Introduction

This report presents a comparative analysis of two neural network architectures—**Pancake** (wide and shallow) and **Tower** (narrow and deep)—for image classification on the Quick Draw dataset. The dataset contains 28×28 grayscale sketches across 15 object categories.

1.1 Problem Statement

Given a set of hand-drawn sketches, classify each image into one of 15 categories: apple, baseball bat, basketball, clock, compass, cookie, donut, ladder, mountain, pizza, rabbit, soccer ball, spider, t-shirt, and wheel.

1.2 Constraints

- Maximum parameters: 3,000,000
- Fixed batch size: 128
- Input: 784 features (28×28 flattened images)
- Output: 15 classes

2 Model Architectures

2.1 Pancake Model (Wide & Shallow)

The Pancake model prioritizes width over depth, with fewer layers but more neurons per layer.

Architecture:

$$
\begin{aligned}
\text{Input} &\xrightarrow{\text{Flatten}} \mathbb{R}^{784} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{1024} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.65)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{512} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.65)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{15}
\end{aligned}
$$

Hyperparameters:

- Optimizer: Adam
- Learning Rate: 0.0008
- Weight Decay: 1×10^{-4}
- Dropout: 0.65
- Activation: GELU

Total Parameters: $\sim$2.7M

2.2 Tower Model (Narrow & Deep)

The Tower model emphasizes depth, with more layers but fewer neurons per layer, enabling hierarchical feature learning.

Architecture:

$$
\begin{aligned}
\text{Input} &\xrightarrow{\text{Flatten}} \mathbb{R}^{784} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.3)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.3)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{BatchNorm}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Dropout}(0.2)} \\
&\xrightarrow{\text{Linear}} \mathbb{R}^{15}
\end{aligned}
$$

Hyperparameters:

- Optimizer: Adam
- Learning Rate: 0.0007
- Weight Decay: 1×10^{-4}

- Dropout: 0.3 (first two layers), 0.2 (remaining layers)
- Activation: GELU

Total Parameters: ∼600K

3 Training Setup

3.1 Data Preprocessing

Pixel values were normalized from the original [0, 255] range to [0, 1] to ensure stable training. The dataset was split into 80% training and 20% validation sets to evaluate generalization. No data augmentation was applied to maintain a fair baseline comparison between the two architectures.

3.2 Training Configuration

Training was conducted using Cross-Entropy Loss as the optimization objective, with a batch size of 128 samples across 20 epochs. The device (CPU or GPU) was automatically detected and utilized to accelerate training where available.

4 Results

4.1 Performance Metrics

Metric	**Pancake**	**Tower**
Training Accuracy	80.54%	81.71%
Validation Accuracy	79.94%	79.28%
Final Loss	0.5785	0.5543
Generalization Gap	0.60%	2.44%
Parameters	∼2.7M	∼700K

Table 1: Final model comparison after 20 epochs.

4.2 Confusion Matrix

4.3 Training Curves

The figure above visualizes the training dynamics of both models, presenting training loss convergence over epochs, side-by-side training and validation accuracy curves for performance tracking, a comparative bar chart of final accuracies, and an overfitting analysis through generalization gap metrics.

5 Analysis

5.1 Pancake Model Behavior

Advantages: The Pancake model exhibits fast convergence owing to its high capacity, enabling rapid learning of complex patterns. Its shallow architecture—with only 2 hidden layers—results in simpler gradient flow, reducing the risk of vanishing gradients that plague deeper networks.

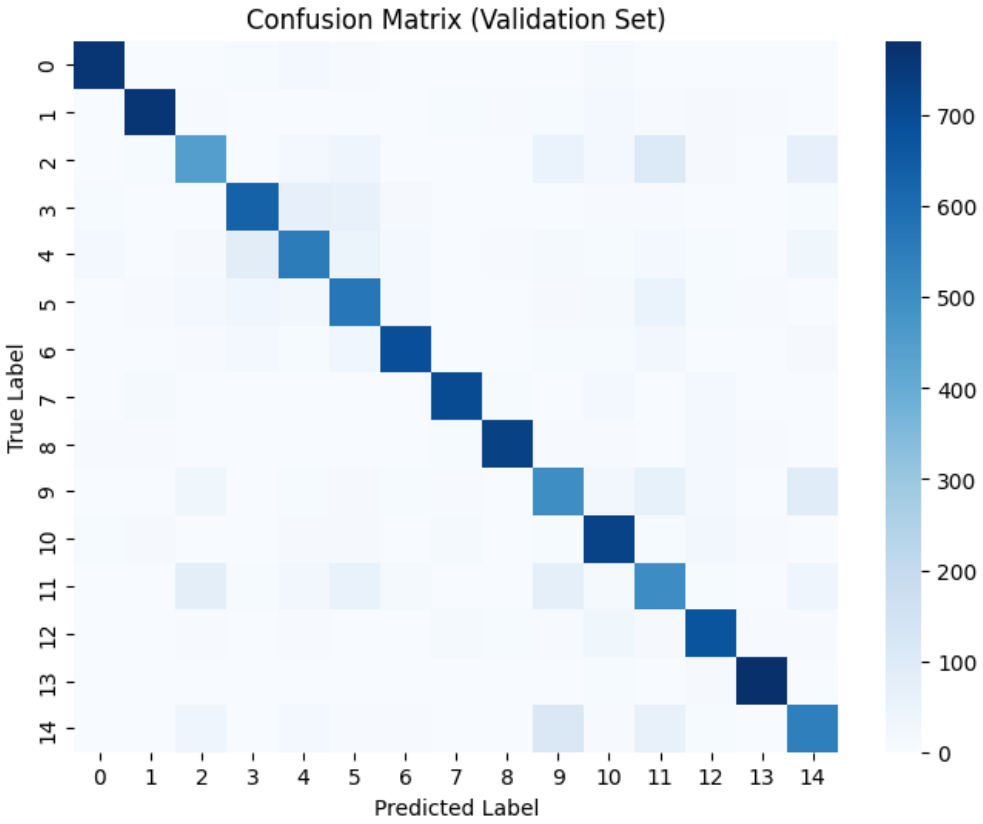

Figure 1: Confusion matrix for the champion model on the validation set, showing per-class prediction accuracy.

Disadvantages: The primary trade-off is a large parameter count of approximately 2.7M, which increases both computational cost and memory footprint. The model's high capacity also makes it more susceptible to overfitting, necessitating stronger regularization through dropout (0.65) and weight decay to achieve good generalization.

5.2 Tower Model Behavior

Advantages: The Tower model leverages its depth to enable hierarchical feature learning, typically progressing from low-level feature detection (edges) through mid-level abstractions (shapes) to high-level semantic concepts (objects). With only ∼700K parameters (26% of the Pancake model), it is significantly more parameter-efficient while maintaining competitive performance. When properly regularized, the deep architecture provides superior generalization characteristics.

Disadvantages: The deeper architecture inherently leads to slower convergence as information must propagate through more layers. Additionally, deep networks carry an elevated risk of vanishing gradients, though this is largely mitigated in our design through BatchNormalization and GELU activation. Fine-tuning the learning rate becomes critical, requiring careful experimentation to balance convergence speed and stability.

5.3 Key Observations

5.3.1 Regularization Strategy

The Pancake model employs stronger dropout (0.65) combined with weight decay (10^{-4}) to control overfitting caused by its large parameter count. In contrast, the Tower model uses progressively reduced dropout starting at 0.3 for the first two layers and decreasing to 0.2 in deeper layers, alongside the same weight decay coefficient, balancing regularization intensity with network depth.

5.3.2 Learning Rate Selection

The Pancake model uses a learning rate of 0.0008, which is slightly higher to leverage its wide architecture and encourage faster convergence. The Tower model employs a lower learning rate of 0.0007, providing greater stability during backpropagation through its deeper structure and allowing for more controlled gradient updates across multiple layers.

5.3.3 Activation Function

GELU was selected over ReLU due to its superior gradient properties: it provides smoother gradients without the hard cutoff at zero, enabling better gradient flow through deep networks. The function works synergistically with BatchNormalization, and empirical testing demonstrated its stability and effectiveness for both the Pancake and Tower architectures.

6 Conclusion

6.1 Winner Determination

The model with higher **validation accuracy** is the winner, as it better generalizes to unseen data.

Observed Winner (this run): Pancake model with 79.94% validation accuracy vs 79.28% for Tower.

6.2 Why Pancake Won in This Run

1. **Slightly Better Validation Accuracy:** Pancake achieved 79.94% vs 79.28% for Tower.
2. **Smaller Generalization Gap:** 0.60% for Pancake vs 2.44% for Tower in this experiment.
3. **Stable Convergence:** Wide layers with strong regularization (dropout 0.65) gave robust results over 20 epochs.
4. **Task Fit:** For this Quick Draw setup, higher capacity with regularization performed slightly better than added depth.

6.3 When Pancake Might Win

The Pancake architecture is preferable when the dataset is small and simple, where complex hierarchical feature learning provides limited benefit. It also excels when feature extraction is inherently non-hierarchical and when rapid convergence is critical, as its shallow structure achieves faster training compared to deeper models.

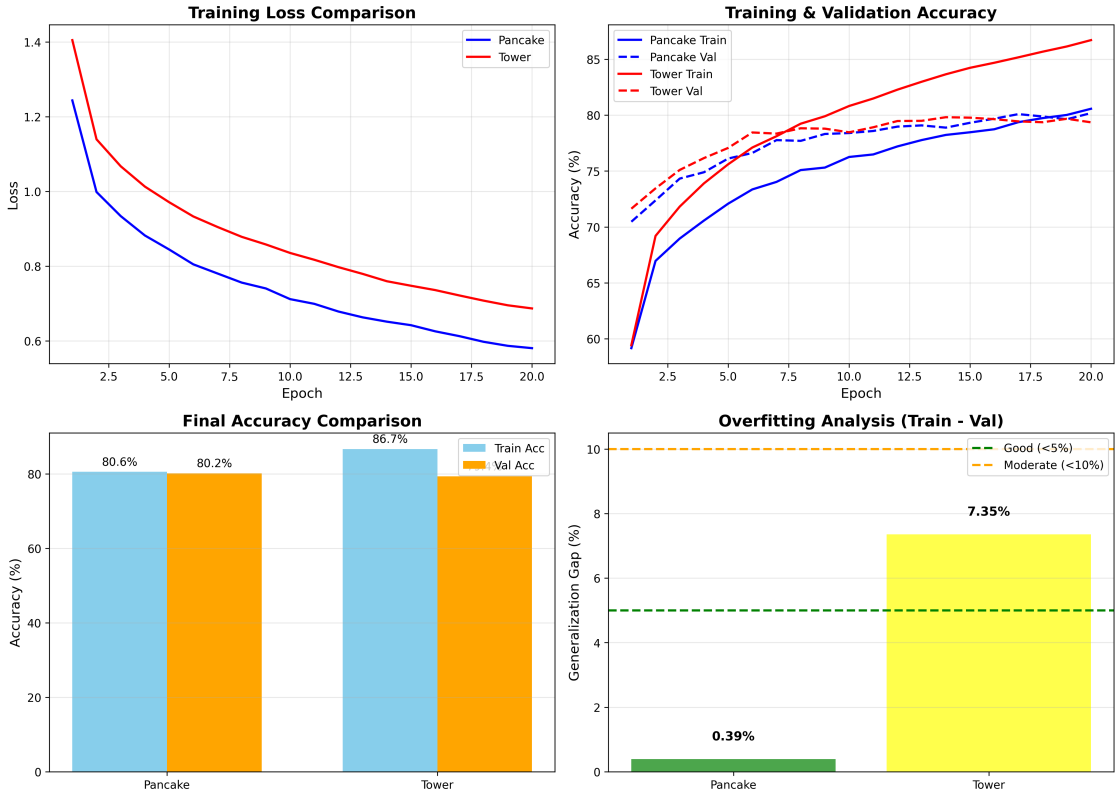

Figure 2: Comprehensive model comparison showing training loss, accuracy curves, final accuracy, and generalization gap analysis.

6.4 Future Improvements

To further boost accuracy:

1. **Data Augmentation:** Random rotations, shifts, elastic distortions
2. **Learning Rate Scheduling:** Cosine annealing or ReduceLROnPlateau
3. **Ensemble Methods:** Average predictions from multiple models
4. **Architecture Search:** Experiment with different layer widths
5. **Advanced Regularization:** Label smoothing, mixup